Computational Economics

Lab: AI Agents, Context, and Skills

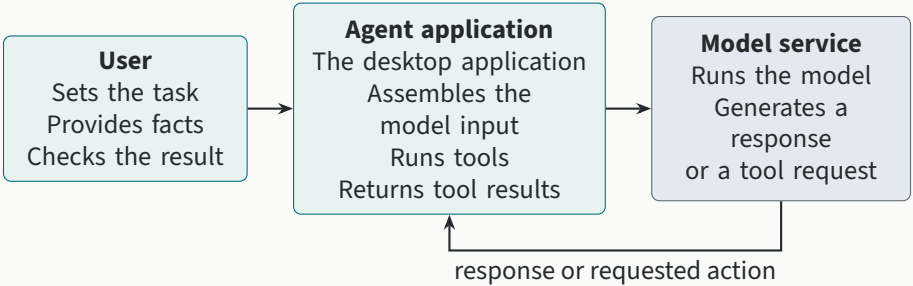
Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lab Overview

- **Give the agent useful information**
  - Clear requests, relevant files, and short results
  - Context limits and caching in everyday use
- **Reuse instructions instead of repeating them**
  - Project rules in `AGENTS.md`
  - Task procedures in skills
- **Keep longer tasks manageable**
  - Small delegated tasks
  - Short notes and a fresh conversation when needed

The Agent Loop: Roles and Responsibilities

An **AI agent** combines a model with an application that can run tools.



- **Tool loop (Lab 4 recap):** the model requests an action; the agent application runs it and returns the result for the model to inspect.
- **Your role:** give a clear goal and relevant facts, then check the outcome. The application and model service carry out the processing.

Tokens: Units of Text

Example sentence: “Please revise my email.”

Before processing text, the model’s tokenizer breaks it into small pieces called **tokens**.

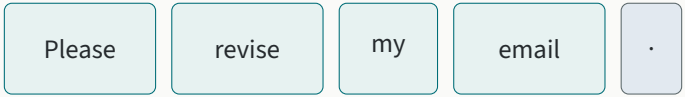
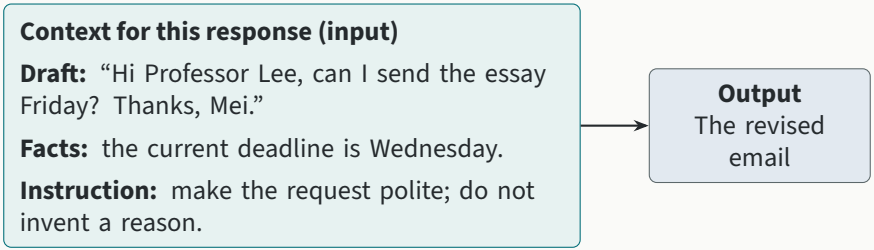


Figure 1: Illustrative split; exact pieces depend on the tokenizer.

- A token can be a word, part of a word, or punctuation. A token count is therefore different from a word count.
- Both the text supplied to the model and the text it generates are counted in tokens.

Context: Input for One Response

Example: the information supplied before the first email revision



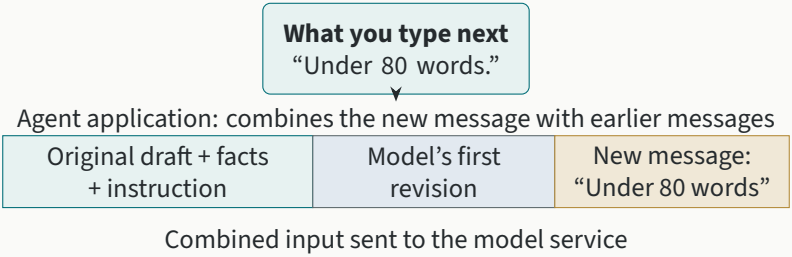
Context is the input available to the model for this response. Here, the draft, facts, and instruction **together** form the example context; the revised email is the output.

The **context window** is the available capacity, measured in tokens. The input and generated response must fit within the applicable limits.

Your Message and the Full Model Input

Example: your first request has already produced an email revision

First request: draft + facts + instruction. Model output: the first revision.



Your new message is short, but the **full prompt sent to the model** is the combined input shown above. That is the context used to produce the next revision.

This example assumes the earlier messages are retained; the agent application can also select or summarize history.

Conversation Growth and Context Capacity

Example: “Under 80 words” followed by “Make it warmer”

Each retained reply and follow-up adds text to the next combined input.

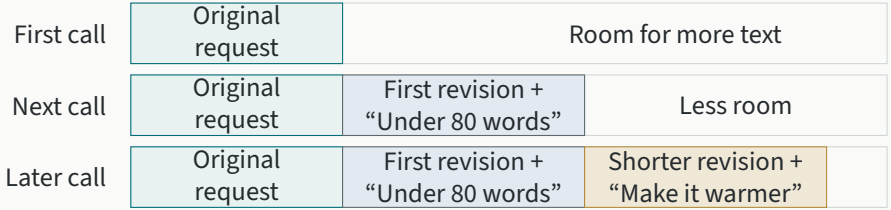


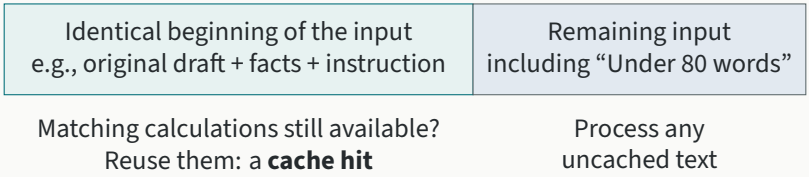
Figure 2: Schematic context capacity; widths are not measured token counts.

- More retained conversation leaves less room for further input and generation.
- Repeated text also means repeated processing. **Caching** can reduce that work; reducing the amount of context is a separate task.

Caching: Reuse by the Model Service

Example: processing the next email revision

The **agent application** assembles the input. The **model service** manages cached calculations and checks whether it can reuse them.



- A **key–value (KV) cache** stores internal calculations for reuse while generating text. Reuse **across calls** is called **prompt/prefix caching** and requires service support.
- A cache hit is a system outcome, not a user command or an agent skill. It can save processing time and, under some pricing plans, input cost.

Cached text **still occupies context space**; caching does not shorten the input.

Cache Hits: Unchanged and Edited History

Example: the same “Under 80 words” follow-up as before

The follow-up needs no special wording. Reuse depends on the beginning of the **full input** assembled by the agent application.

Append the follow-up

Keep the first request unchanged.
Send: “Under 80 words.”

The original beginning can still match the cached input.

Rewrite an earlier message

Change “make the request polite” to “rewrite the email politely.”

The match with the old input stops at the first changed token.

- **User practice:** append new requirements; avoid needless edits to correct history. Correct mistakes whenever needed, even if this reduces reuse.
- **Model service:** may reuse a matching prefix if caching is supported and an eligible entry exists. This short email may be too small to qualify.

AGENTS.md: Instructions Across Tasks

Example: revising more than one email

Save standing rules in AGENTS.md

Use a polite, concise tone.
Preserve names, dates, and facts.
Do not invent reasons.

Put today's details in the request

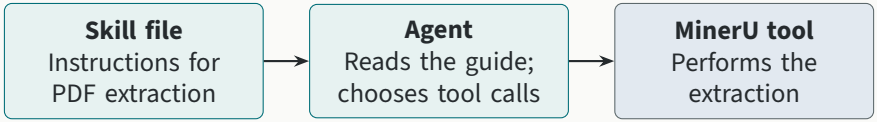
Recipient: Professor Lee.
Purpose: an extension to Friday.
Length: fewer than 80 words.

- **Agent application:** where supported, it loads applicable `AGENTS.md` instructions alongside the current request.
- **Your practice:** keep these rules short and current; supply the draft and task-specific details each time. Follow the agent application's file-location rules.

Agent Skills: Instructions for a Repeated Task

Instructor's example: my PDF-extraction workflow

For each research PDF, I need the same sequence: run extraction, check the output files, and report the results. I saved that procedure in a **skill**.



My file is `Research/agent-skills/skills/pdf-extraction/SKILL.md`.

- **Writing a skill:** save a repeatable procedure with inputs, steps, checks, and expected outputs.
- **Using it:** supply the PDF path and desired destination. The agent needs access to the named tool; the skill file does not install it.

SKILL.md: Description and Procedure

Two parts of my PDF-extraction skill

1. Description: helps the agent choose the skill (actual metadata)

```
---
name: pdf-extraction
description: >-
  Run local MinerU to extract PDFs into Markdown,
  structured data, and assets, with structural checks only.
---
```

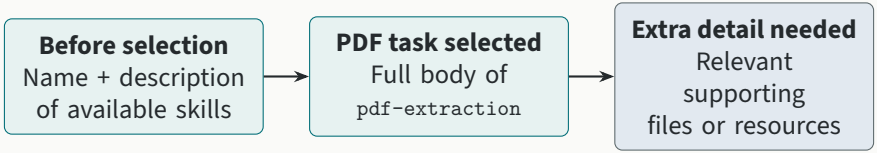
2. Main body: guides the agent's actions (summarized)

1. Choose the output location and run the extraction tool.
2. Check output files, page coverage, and linked images.
3. Return paths and check results; identify anything unverified.

Your skill: specify when to use it; write concrete steps and checks.

Progressive Disclosure: Loading a Skill in Stages

Example: what enters context when PDF extraction is needed



- **Agent loading:** **progressive disclosure** makes names and descriptions visible first; the body is read on activation, supporting files only as needed.
- **Skill design:** keep the description brief and specific. Put core steps in the body; link lengthy reference material separately.
- **In your request:** name the needed skill rather than pasting every guide. An email task need not load the extraction procedure.

Skill Selection: From Request to Procedure

Example: selecting the PDF-extraction guide

A matching task

"Extract `paper.pdf` to Markdown."

The agent may recognize that the task matches the skill's description.

An explicit selection

"Use `pdf-extraction` on `paper.pdf`."

The request names the intended skill.

- **Agent sequence:** select the skill → read its body → use tools and check results. Selection alone is not execution.
- **Your request:** name the skill when you want a specific procedure. If discovery is unavailable, give its path: "Read this `SKILL.md`, then follow it."

Tool Results: Files and Returned Information

Example: what my extraction skill returns (illustrative)

Full outputs on disk

Extracted Markdown
Images and structured data
Detailed execution log

The original PDF is preserved.

Report returned to the agent

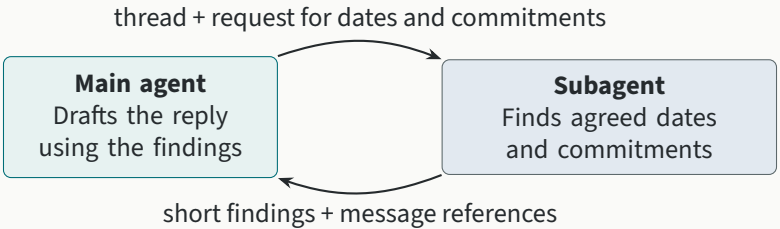
File: `notes/paper.md`
Pages accounted for: 12 of 12
Referenced images missing: 1
Image check: failed

- **Agent input:** the short report enters the next context. This skill keeps the full extracted content and logs on disk during extraction.
- **Your check:** ask for output paths, coverage, and failures. Here, one missing image needs follow-up; “12 of 12 pages” is not an all-clear.
- **Content review:** request it separately when needed. File checks do not establish that the extracted equations are correct.

Subagents: Delegating a Separate Review

Example: drafting a reply to a long email thread

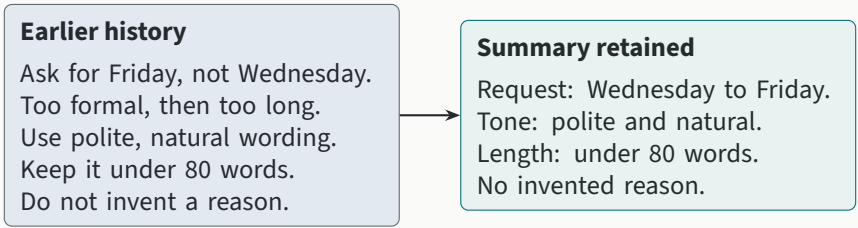
A **subagent** handles a separate task in its own context and reports back (where supported).



- **For a long thread:** supply the thread and a focused task: “Find agreed dates and commitments; return message references.”
- **Before drafting:** ask the main agent to verify the findings against those references.
- **For a short email:** use one agent; delegation adds work.

Compaction: A Shorter Conversation History

Example: several revisions of the email to Professor Lee



- **Agent application:** **compaction** replaces history with a shorter summary, automatically or on request where supported.
- **Before compaction:** save the current draft and important facts. A summary can omit details or keep an old mistake.
- **When continuing:** check dates, tone and length; restore missing requirements.

Fresh Sessions: Saved Notes as New Input

Example: finishing the email in a new conversation

Agent application: adds file contents to context when they are read. A fresh session need not contain the full earlier conversation.

```
# RUN_NOTES.md (illustrative)
Draft: drafts/extension_email.txt
Recipient: Professor Lee; request Wednesday to Friday.
Style: polite and natural; under 80 words; no invented reason.
Next: check the final wording and word count.
```

- **Before restarting:** save the latest draft, current requirements and next step.
- **First request:** "Read `RUN_NOTES.md` and the saved draft, then check the wording and word count." Attach the files if the agent application cannot read their paths.

Practical Instructions for Agent Tasks

1. **State the task clearly.** Provide the input, desired output, and constraints: audience, tone, length, or file destination.
2. **Reuse the right instructions.** Keep standing rules in `AGENTS.md`; name a suitable skill for a recurring procedure. Make the files available to the agent.
3. **Keep the context relevant.** Supply needed material and request short reports. Delegate only a clearly bounded task. Caching does not free context space.
4. **Check the result.** Compare it with the request. Inspect changed facts, generated files, completed checks, and unresolved errors.
5. **Save enough to continue.** Record current decisions, output paths, and the next step before compaction or a fresh conversation.

References and Further Reading

1. **Hugging Face, Agents Course: the agent loop.**
2. **Hugging Face, Tokenizers.**
3. **Hugging Face, How caching works.**
4. **vLLM contributors, Automatic Prefix Caching.**
5. **Anthropic (2025), Effective context engineering for AI agents.**

References and Further Reading

6. **Anthropic, Prompt caching.**
7. **Agent Skills contributors, Agent Skills specification.**
8. **AGENTS.md contributors, Project instructions for coding agents.**
9. **CarpeDiem (2026), What should I do if the model feels less capable? [Video, Chinese]**